FYP Idea Selection and Proposal Readiness Guide

(For all Final Year Project groups under the FAST School of Computing)

Core standard
Every FYP, regardless of stream, domain or supervisor, must demonstrate six connected elements: (1) research-informed problem understanding; (2) a clearly justified complex computing problem or complex computing activity; (3) a coherent designed and implemented system that runs in a realistic deployment environment; (4) systematic testing and evaluation; (5) the engineering documentation set that forms the project report; and (6) traceable team and individual ownership. The idea should also state a relevant SDG or thematic alignment as broader context. This alignment is not a substitute for CCP, contribution, feasibility or evaluation. An idea may enter through published research or through an observed real-world problem, but both entry points are judged against this same standard. These are not separate tracks; they form one common academic mechanism for all FYPs.

Important policy note: Permission to use any external tool, service, model, API, dataset, library, template, or third-party component is governed by current university policy on academic integrity, ethics, plagiarism, disclosure, licensing, copyright, privacy and security. This guide neither grants nor withholds that permission. What this guide defines is the academic standard that applies to whatever use is permitted.

External components are expected, not exceptional: Modern computing practice is built on frameworks, libraries, services, pre-trained models, hosted APIs and templates, and their use in a final year project is normal and expected. The boundary is not whether an external component is used, but what it supplies. An external component may support the project. It may not supply the complex computing problem the group claims to have solved. Section 14 sets out how that boundary is classified and judged.

Regardless of how any part of the work was produced, students remain individually responsible for the reasoning, design, implementation, verification and defense of what they submit. Authorship of keystrokes is not the test a panel applies; understanding, justification and the ability to modify the work under questioning are. Permitted use that is properly disclosed and acknowledged is not penalized. Undisclosed use, or work that cannot be explained by the member who claims it, is treated as an integrity or ownership failure.

How to use this guide:
Sections 1 to 17 apply to every FYP without exception and should be read first. Annex A adds guidance for groups whose idea begins from published research. Annex B adds guidance for groups whose idea begins from an observed real-world or everyday problem. Annex C shows worked examples of complex computing problems formulated at three levels of maturity, and applies to both streams. The annexes assume the common sections and do not repeat them.

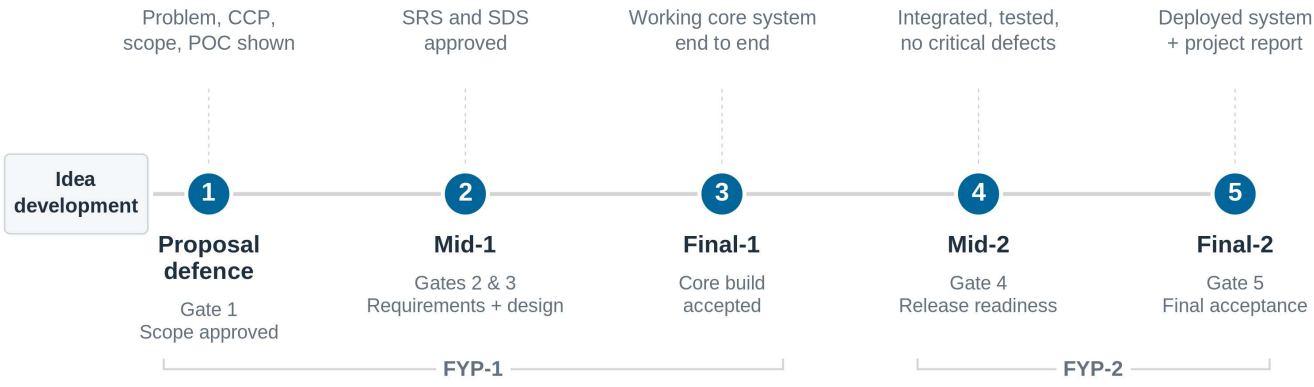


Figure 1. The FYP at a glance — from idea development through five evaluation stages to a deployed system and project report.

1. What kind of FYP should you select?

A Final Year Project (FYP) is the capstone of the computing degree. It should not be selected merely because an application can be built around it. A suitable idea begins with a meaningful problem, develops a defensible technical contribution, and provides enough evidence for the panel to judge both the quality of the product and the learning of the students.

- Every FYP must be research-informed and evaluated, not only implemented.
- Every FYP must produce a working, reproducible computing artefact that can be deployed and operated outside the development machine and not only a literature review, a notebook of results or a recorded demonstration.
- Every FYP must produce the engineering documentation set that culminates in the project report, and the idea must be specific enough to be written down in that form.
- An industry-collaborative FYP must still satisfy every academic requirement, even when the client mainly values delivery.
- A data, AI, security, networking, HCI, IoT, software engineering or infrastructure FYP is judged by the same core mechanism, although the evidence will differ by domain.

1.1 Two Entry Points, One Standard

The School recognizes two entry points into an FYP idea. Stream A begins from published research. Stream B begins from an observed real-world or everyday problem. The stream describes where the idea came from and not how difficult it is, not how prestigious it is, and not how it will be graded. Both streams converge on the same obligations: a defensible complex computing problem, a working deployable system, systematic evaluation, a complete documentation set and traceable individual ownership. A group should declare its stream at proposal stage so that the panel can ask the right questions and may change it later only with supervisor approval.

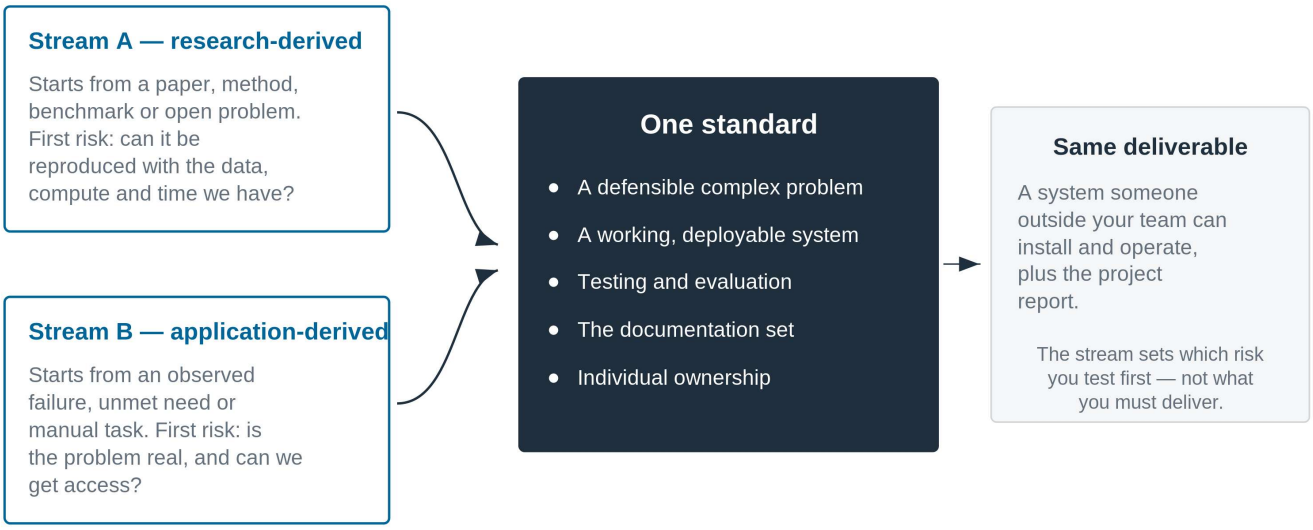


Figure 2. Two entry points, one standard. The stream determines which risk you test first, not what you must deliver.

Aspect	Stream A — research-derived	Stream B — application-derived
Starting point	One or more peer-reviewed papers, a published method, a benchmark, a dataset or an open problem stated in the literature.	An observed workflow failure, an unmet stakeholder need, or an everyday task that is currently handled manually or badly.
Where the gap comes from	A stated limitation, an unexamined assumption, an untested setting, a missing comparison or an unaddressed constraint in the published work.	Evidence from stakeholders, observed practice, product documentation, standards and prior FYPs showing what remains unsolved or inadequately solved.
Primary risk to test early	Whether the method can be reproduced, adapted or even run within the data, compute, license and time budget actually available to the group.	Whether the problem is real, whether stakeholders will engage, and whether the required data, access, permissions or integration can be obtained.
Typical contribution	Transparent reproduction, adaptation to a new context or constraint, comparison of methods, failure analysis, or engineering a published method into an operable system.	Problem or requirement formulation, system and architecture design under real constraints, workflow or accessibility improvement, or a curated domain resource.

Aspect	Stream A — research-derived	Stream B — application-derived
Most common failure	Re-running someone else's code without understanding it, and finishing with results but no system that anyone could operate.	Building a competent application around a problem that never required computing depth, and discovering at defense that no CCP can be identified.
What the panel will press on	Can you explain, modify and defend the method without the code in front of you? What is your delta over the paper? Does it become a real system?	Where exactly is the complexity? What would a routine implementation have got wrong? What evidence shows this problem is real?

Both streams end in the same place
A Stream A project that produces excellent results but no operable system has not met the standard. A Stream B project that produces a polished application with no research basis, no defensible complex computing problem and no evaluation beyond a demonstration has not met the standard either. The stream determines where a group starts looking and which risk it tests first. It does not reduce what the group must deliver.

1.2 Working, Deployable System

An FYP is not complete when the code runs on the developer's laptop. The final artefact must be installable and operable by someone other than its authors, in an environment that reasonably represents its intended use. This is a requirement of engineering competence rather than of commercial success: the panel assesses whether the system is deployable, documented and defensible, not whether it has users or revenue. Groups should nevertheless be able to say who would use the system, what adoption would require and what stands in the way. That thinking almost always improves the requirements, and it frequently exposes the constraints where the real computing complexity lives.

Deployment readiness dimension	What should be true by Final-2
Runs outside the development machine	The system can be installed and started from documented steps, on a clean environment, by someone who did not build it.
Realistic operating environment	The deployment target is stated and justified e.g., cloud, on-premises, container, edge device, mobile, offline or embedded etc., and the system is deployed there rather than only demonstrated locally. Deployment to the real target is expected wherever it is technically and legally feasible; where it is not, the group deploys to the closest defensible substitute and states what remains untested.
Configuration and data	Configuration, credentials, secrets and seed or sample data are handled deliberately rather than hard-coded, so that the system starts from a known, reproducible state.
Non-functional behavior	Performance, capacity, reliability, security, privacy and accessibility behavior appropriate to the problem is stated as measurable criteria and tested, not asserted.
Failure and recovery	Known failure modes, degraded behavior, and recovery or fallback paths are identified and demonstrated, including what happens when an external dependency is unavailable.
Handover	Source, build and run instructions, dependencies, licenses, known limitations and future work are packaged so that another party could continue, evaluate or operate the project.
Adoption context	Intended users, the realistic path to use, and the principal barriers such as cost, data access, integration, regulation, trust or maintenance are identified honestly. This is assessed as engineering judgement, not as commercial achievement.
Evidence of deployment	A record of the deployed system: where it runs, how it was installed, who can access it, and what it cost to stand up. A screenshot of a local build is not deployment evidence.

Deployability is required; commercial success is not
Groups are not expected to launch a product, acquire users or demonstrate revenue. They are expected to build something that could credibly be handed to a real user or operator. Where a project genuinely cannot be deployed into its intended environment — for example where clinical, legal, safety or regulatory approval would be required. The group must deploy into the closest defensible substitute and state explicitly what remains untested, and why.

1.3 SDG or Thematic Alignment

Every group should identify one relevant UN Sustainable Development Goal (SDG) or one relevant thematic area such as health, education, environment, smart cities, innovation, accessibility, safety, public services, industry or community welfare. The link should be explained in two or three sentences by connecting the project problem, stakeholders, and expected contribution to that theme.

Do not use SDG or thematic alignment to decorate a weak idea. A broad label such as “education” or “innovation” does not prove that the project is complex, useful or academically strong. The alignment only gives wider context; the project must still defend its problem, CCP or complex activity, contribution, POC, evaluation and ownership.

Field	What students should write
Primary SDG or thematic area	Select one main SDG or theme that naturally matches the project problem.
Connection to problem	Explain which stakeholder problem or consequence creates this alignment.
Boundary	State clearly that the SDG/ theme is context and not the claimed technical contribution or CCP.

2. Research and Development is Embedded in Every FYP

Research in an undergraduate FYP does not always mean inventing a new algorithm or publishing a paper. It means that the project is informed by credible evidence rather than assumptions. The group must study what already exists, understand relevant technical knowledge, use that understanding to make decisions, and define how the proposed solution will be evaluated.

R&D element	What it means in a student FYP	Evidence expected at proposal stage
Solution landscape	Study existing commercial products, open-source systems, prior FYPs, manual workflows and relevant standards.	A comparison that identifies what is already solved, what remains weak, and why the proposed work is not a copy.
Technical foundation	Study relevant methods, architectures, algorithms, datasets, protocols, standards, design principles or domain knowledge.	A focused review showing which sources influence the proposed approach and which alternatives were considered.
Reproduction, adaptation or experimentation	Where suitable, reproduce or adapt a published method, benchmark alternatives, or test a technical assumption.	A plan for implementation/experimentation and a clear statement of what will be reproduced, changed or compared.
Evaluation and learning	Define evidence that will show whether the solution is correct, useful, reliable or better suited to the problem.	Baselines, test scenarios, metrics, user studies, benchmarks or acceptance criteria appropriate to the project.

State-of-the-art work is welcomed: two forms of engagement with recent, high-performing research are particularly encouraged. The first is implementing a state-of-the-art method as a deployable product. Published work very often stops at a reported result on curated data, and the engineering required to turn that method into something a user or operator can actually run — under real input, real latency, real cost, real failure and real deployment constraints — is substantial, defensible and frequently where the complex computing problem actually lives. The second is improving the methodology of a state-of-the-art paper at a gap the group has identified through its own study of the literature, and then demonstrating that improvement against the original method as a baseline. Both are strong bases for an FYP. Both must still end in a working, deployable system and a defensible evaluation.

What is not required in every FYP: novelty is not a compulsory claim. A group is not obliged to invent a new algorithm or to beat the best published number, and Section 4 sets out several other contribution types that are fully acceptable when they are substantial and properly justified. What is required at every level of ambition is understanding. Implementing or improving a state-of-the-art method without being able to explain its mechanism, its assumptions and its failure modes is not acceptable, however impressive the result appears.

The selected sources should be current and authoritative for the topic, while seminal papers, established standards and official documentation may also be necessary. What matters is that the research changes or justifies the project's problem formulation, design or evaluation. A literature section that has no visible effect on the project is not sufficient.

3. Understanding a Complex Computing Problem (CCP)

The Seoul Accord defines a complex computing problem as a computing problem having some or all of a set of recognised characteristics. It does not require every characteristic, and it does not prescribe a fixed number that must be ticked. However, a credible CCP claim should be central to the project and normally be supported by several meaningful characteristics, not by one superficial statement.

Seoul Accord characteristic	Question for your idea	How to interpret it?
1. Conflicting requirements	Are there important technical, computing, organizational, legal, ethical, economic or user requirements that pull the solution in different directions?	Example: accuracy versus latency, privacy versus personalization, security versus usability.
2. No obvious solution or depth of analysis	Is there no standard answer that can simply be copied? Does the group need conceptual thinking, modelling or comparative analysis?	A standard CRUD workflow normally does not satisfy this characteristic.
3. Depth of knowledge	Does the solution require in-depth computing or domain knowledge and reasoning based on established principles?	The group should identify the computing and domain knowledge it must learn and apply.
4. Unfamiliar issues	Does the project involve issues that are not frequently encountered in routine development?	Unfamiliarity should come from the problem, constraints or analysis, not from choosing an unfamiliar framework.
5. Beyond standard practice	Is the central problem outside what can be solved completely through standard procedures, templates or ordinary professional practice?	Using a standard framework is acceptable, but it should not solve the core challenge by itself.
6. Diverse stakeholders	Are there stakeholder groups with significantly different needs, authority, risks or success criteria?	The project must show how these needs affect requirements and design.
7. Significant consequences	Can poor performance, wrong decisions, insecurity or failure have important consequences?	The evaluation and safeguards should match the seriousness of the consequences.
8. Interdependence	Is the problem high-level, with multiple interacting sub-problems whose behavior affects the whole system?	A large number of independent screens is not the same as interdependent technical sub-problems.
9. Ill-defined requirements or cause	Is part of the requirement, root cause or correct solution initially unclear and in need of investigation?	Students should show how research, stakeholder study, experiments or a POC will reduce this uncertainty.

Do not manufacture complexity
A project should not add unnecessary technologies, modules or difficult features merely to appear complex. The CCP should arise naturally from a real problem and should lead to meaningful analysis, design trade-offs and evaluation. Artificial complexity increases risk but does not improve academic quality.

Annex C shows twelve domains formulated at three levels of maturity, together with the characteristics each formulation exhibits. For most groups the contrast between a weak and a strong formulation of the same problem is more instructive than the table above.

Does a CCP make a project more interesting?

A CCP does not automatically make an idea attractive. It makes the idea academically defensible when the complexity is connected to a worthwhile problem. A project becomes interesting when students must make real decisions, resolve trade-offs, test uncertain assumptions and produce evidence. The purpose of CCP is therefore not to make the project look difficult; it is to explain why the project deserves capstone-level study and cannot be reduced to routine implementation.

Complex computing activity is also relevant

The Seoul Accord separately describes a complex computing activity or project through characteristics such as diverse resources, significant interactions among technical and contextual issues, creative use of knowledge, significant consequences, and work that extends beyond previous experience using principles-based approaches. A project may therefore be defensible through one central CCP and the complex activity required to solve it. Students should not claim several unrelated CCPs only to inflate scope.

4. Contribution: What Value Will Your FYP Add?

Contribution means the meaningful value created by the project beyond repeating what already exists. Novelty may strengthen a project, but novelty should not be treated as a compulsory claim of invention. A rigorous reproduction, adaptation, integration under difficult constraints, evaluation, or workflow improvement may be a valid contribution when it is substantial and properly justified.

Possible contribution type	Examples	What must be shown
Problem or requirement contribution	A clearer formulation of a neglected sub-problem; resolving an ill-defined workflow or stakeholder conflict.	Evidence that the problem is real, not already adequately solved, and that the formulation changes the solution.
Method or algorithm contribution	Reproducing, adapting, combining or comparing relevant techniques for a new context; improving the methodology of a state-of-the-art method at an identified gap; or engineering a state-of-the-art method into a deployable system.	A justified baseline — normally the original method — transparent adaptation, and evidence of performance, cost or limitations under the group's own evaluation.
System or architecture contribution	A reliable, secure, offline, distributed, edge, scalable or interoperable solution under meaningful constraints.	Why ordinary architecture is insufficient; key trade-offs, interfaces, risks and validation.
Data or resource contribution	A curated dataset, annotation process, benchmark, ontology, simulator or domain-specific data pipeline.	Quality controls, ethical/legal handling, documentation and how the resource supports the problem.
Human/workflow contribution	Human-in-the-loop decision support, accessibility, explainability, safer workflow or improved coordination.	Stakeholder evidence and evaluation of the actual workflow, not only interface appearance.
Evaluation contribution	A defensible benchmark, failure analysis, user study, comparison method or validation protocol.	Why the evaluation is suitable and what conclusions it can and cannot support.

Contribution statement

Use a precise sentence: "Existing approaches perform X, but remain weak under Y condition. We will contribute Z through A, and evaluate it using B." Avoid claims such as "we will use AI", "we will add RAG", "we will make it local", "we will improve the UI", or "we will combine many features" unless the technical contribution and evaluation are clearly explained.

5. Scope for the Approved Group Across Two Semesters

An FYP should be one coherent project, not a collection of loosely connected individual mini-projects. Scope is not judged mainly by the number of screens, technologies, features or estimated coding hours. It is judged by the depth

of the problem, the amount of reasoning and engineering required, the quality of the evidence, and whether every registered member can make a meaningful and accountable technical contribution.

Scope principle	Practical meaning
One shared central problem	All members should be able to explain the same problem, contribution, architecture and evaluation story.
Meaningful primary ownership	Each member should have a substantial technical responsibility that includes decisions, implementation and validation. A member should not be limited to slides, documentation, routine UI or manual testing.
Interdependent work	Responsibilities should connect through defined interfaces, shared data, integration or common evaluation. The project should not become separate individual submissions.
Shared engineering responsibilities	Architecture, integration, quality assurance, ethics/security, repository discipline and final defense remain team responsibilities even when one member leads a part.
Balanced depth, not identical tasks	Contributions need not be equal in type. One member may lead a data/model pipeline, another a core system or service, and another an interaction/deployment/evaluation subsystem, provided each contribution is technically meaningful and the workload is reviewed over time.
Evolving division of work	Initial ownership can change as the design evolves. The team should maintain a task/milestone record and be prepared to justify changes.

A useful scope test: If one member were removed, would a meaningful technical responsibility and its evidence disappear? If the answer is no for a member, the proposed division is probably too shallow. This is a diagnostic question, not a rule that each member must build a separate module.

5.1 Checkpoints: The Five Evaluation Stages

An FYP is assessed at five points. The proposal defense is the first, followed by Mid-1 and Final-1 in the first semester, and Mid-2 and Final-2 in the second. Groups should test a candidate idea against this trajectory before committing to it. An idea that cannot show demonstrated feasibility by the proposal defense, or cannot reach a working core system by Final-1, is too large, too uncertain or too dependent on resources the group does not yet have.

These five points correspond closely to the approval gates defined in the School's project documentation standard, and the table below names both. The technical column states what should exist in the project itself; the documentation column states the state of the written record. Detailed assessment criteria and weightings are defined by the FYP evaluation forms, and the content and format of each document by the documentation standard; where this table and those documents differ, they take precedence.

Stage and gate	What should exist in the project	Documentation state
1. Proposal defense Gate 1 — Scope approved	A defined problem, an evidenced gap, a justified CCP, a contribution statement and an evaluation plan. A demonstrated POC or technical spike testing the highest-risk assumption. Committed scope separated from optional extensions, with primary technical ownership allocated to each member.	Scope of work approved — see 5.2 on who approves it. Project proposal and initiation plan complete. First version of the timeline, milestones and deliverables, with a named approver and an acceptance basis for each deliverable. Initial RAID register opened for risks, assumptions, issues and dependencies.
2. Mid-1 Gate 2 — Requirements baseline Gate 3 — Design baseline	Requirements baselined with stakeholders. Architecture chosen and justified against the alternatives considered, with key design decisions and their rationale recorded. Data, access, licenses and ethical clearance secured. Wireframes or a clickable prototype validated with real users. The riskiest subsystem prototyped.	Software requirements specification approved: requirements uniquely identified, testable and prioritized, with at least one measurable non-functional requirement for every quality attribute that matters. Software design specification approved, covering components, data design, interfaces, security, deployment topology and failure handling. Traceability matrix opened. API and third-party dependency register started.

Stage and gate	What should exist in the project	Documentation state
3. Final-1 Exit gate — Core build accepted	A working core system covering the primary scenario end to end, built against the approved design. Integration path proven rather than assumed. The comparison baseline measured. Deployment target selected and a first deployment attempted.	Traceability matrix populated for all committed requirements, showing requirement to design to test coverage. Test, deployment, rollback and handover plan drafted, including defect severity definitions and release criteria. Timeline revised against actual progress, with any scope change processed under 5.3.
4. Mid-2 Gate 4 — Release readiness	Committed scope implemented and integrated. System testing complete. Non-functional criteria measured against their stated targets rather than asserted. Evaluation running against the baseline. Failure and edge cases exercised deliberately.	System test report with results and a defect list by severity: no open Critical defects, and every open High defect carrying an approved disposition. Deployment and rollback plans approved and rehearsed rather than described.
5. Final-2 Gate 5 — Final acceptance	A deployable, documented system installed and operated in the target environment — or a defensible substitute — by someone outside the team. Acceptance testing completed with the supervisor or client. Completed evaluation with results, limitations and failure analysis. Every member able to defend their technical ownership.	Complete project report: all documents at their approved version, acceptance matrix signed off, and a handover package containing source, build and run instructions, dependencies, licenses, credentials handling, known limitations and future work. Any remaining items recorded in a signed punch list with owners.

Every document in the set carries a document control block recording version, date, author and status, and moves from draft, through review, to approved baseline. Groups should version their documents rather than overwrite them. The panel should be able to see how the requirements and the design evolved across the two semesters, and why.

This mapping is proposed for review
Two judgements in this table should be confirmed before the guide is issued. First, both the requirements baseline and the design baseline are placed at Mid-1, which follows the documentation standard's rule that design is baselined before implementation — but it also determines how much implementation FYP-1 demands. Second, the documentation column is a proposed sequencing of the School's document set and should be checked against the FYP evaluation forms. Groups should in all cases follow the stage requirements published for their cohort.

5.2 Who Acts as the Client

The School's documentation standard is written for a client-and-vendor relationship: the scope of work is issued by a client, and the development team responds to it. Final year projects rarely have that relationship, so the role must be assigned explicitly at proposal stage rather than left ambiguous.

Project situation	Who acts as client
Industry-collaborative project	The partner organization acts as client and approves the scope of work. The supervisor remains the academic approver, and the client's satisfaction does not by itself satisfy the academic requirements.
Supervisor-proposed project	The supervisor acts as client for scope approval while remaining the group's supervisor. The group should record which decisions were taken in which capacity.
Group-originated project (most Stream B ideas)	The group writes the scope of work on behalf of a named, real stakeholder it has actually consulted, and the supervisor approves it. An invented client is not acceptable.
Research-derived project (most Stream A ideas)	The intended operator or user of the resulting system acts as the notional client. The scope of work states who would run the system and what they would need from it.

In the commercial model the client supplies a prioritized list of functional and non-functional requirements and the vendor implements it. In an FYP, the group must discover the requirements itself — through stakeholder study, the

solution landscape and its own analysis — because that discovery is a substantial part of the academic work being assessed. A group that is handed a requirements list has been handed the part of the project that carries the marks. Where an industry partner does supply requirements, the group must still establish the problem, the gap, the CCP and the evaluation independently, and should state plainly which requirements were given and which it derived.

5.3 Change Control after Approval

Approval at the proposal defense baselines the scope. Projects change after that — a design proves unworkable, data turns out to be unavailable, an assumption fails — and changing course in response to evidence is a sign of good engineering rather than of failure. What is not acceptable is silent change: quietly dropping a committed requirement, substituting an easier problem, or expanding scope without assessing the consequence.

Once a document reaches its approved version, any change affecting scope, schedule, architecture, security, data handling or acceptance criteria should be recorded and approved by the supervisor before it is acted upon. The record need not be elaborate — what changed, why, what evidence prompted it, what it affects and who approved it — but it must exist, and the panel should be able to trace the final system back to the approved proposal through it.

Change requiring a recorded decision	Why it matters
Removing or deferring a committed requirement	The committed scope is what the project was approved against. Deferral is acceptable when justified and recorded; silent removal is not.
Changing the core method, architecture or technology	The proposal was approved on a stated technical approach. Changing it changes the CCP claim, the risk profile and often the contribution itself.
Changing the deployment target or relaxing a non-functional target	Non-functional targets form part of the acceptance basis. They are not aspirations to be quietly lowered once they prove difficult.
Changing the evaluation baseline, metrics or success criteria	Adjusting the measure after seeing the result invalidates the evaluation. Such changes must be justified and made before the results are known.
Reassigning primary technical ownership between members	Individual assessment depends on traceable ownership. Reallocation is permitted, but must be recorded when it happens rather than reconstructed at the end.
Adding scope beyond the committed core	Extension is welcome once the core is secure, but it must not place the committed scope at risk, and it should be recorded as an extension rather than absorbed.

Change is expected; undocumented change is not
A group that revises its approach on evidence and records the decision demonstrates engineering judgement, and should present that revision as a finding. A group whose final system quietly differs from its approved proposal, with no record of when or why it diverged, has lost the traceability on which its own report, its acceptance matrix and its individual assessment all depend.

Scope sufficiency questions

- Is the central problem complex enough to require analysis and design, rather than only routine implementation?
- Can the group identify meaningful technical decisions, risks and validation work across both semesters?
- Does each member have a defensible technical role with primary ownership and shared integration responsibility?
- Can the team demonstrate feasibility at the proposal defense, baseline requirements and design by Mid-1, reach a working core system by Final-1, and deliver an integrated, evaluated and deployed system by Final-2?
- Is there a clear minimum committed scope, with optional extensions separated from the core?
- Can the project be evaluated with evidence beyond a successful demonstration?

6. Can This Idea Be Specified?

An approved idea does not go straight to code. It becomes a scope of work, a proposal and initiation plan, a timeline of milestones and deliverables, a software requirements specification, a software design specification, a requirements traceability and acceptance matrix, and a test, deployment and handover plan. Together these form the project report, and they are the primary written evidence of the group's engineering work. An idea that cannot be written down in this form is not ready for approval, however appealing it sounds in conversation.

The specifiability test is straightforward. At proposal stage the group should already be able to sketch, in outline, what its requirements, design and acceptance criteria would look like. Precision is not expected this early and requirements will change; what is expected is that the group can name concrete functional requirements, state at least one measurable non-functional requirement, and describe the evidence that would prove a requirement has been met. Vagueness at this stage is not a documentation problem — it is a sign that the problem has not yet been analyzed.

Question to answer before approval	Why it decides readiness
Can you name at least five concrete functional requirements, each in the form "the system shall…"?	If requirements can only be expressed as themes — "smart", "AI-powered", "user-friendly", "automated" — the problem has not been analysed yet.
Can you state at least one measurable non-functional requirement?	Performance, availability, accuracy, security, privacy, accessibility and deployment targets must be expressed as numbers or observable criteria, not as adjectives.
Can you name the actors and the main data the system will hold?	If the group cannot list who uses the system and what it stores, neither the design nor the privacy, security and ethics analysis can begin.
Can you state an acceptance criterion for your hardest requirement?	Every requirement must eventually trace to a design element and a test. A requirement that cannot be tested cannot be accepted, and should not be committed to.
Can you draw a first architecture on one page?	Major components, interfaces, data flows and external dependencies should be sketchable at proposal stage, even though they will change.
Do you know which parts are most likely to change?	Naming the uncertain parts is a sign of understanding, not of weakness. A group that believes nothing will change has not yet thought hard enough about the problem.

A one-sentence test for vagueness

Complete this sentence for the hardest part of the project: "The system shall <do something specific> for <a named actor>, and we will know it works when <observable evidence>." If the group cannot complete it, the idea needs more work before it is taken to defense. If the group can complete it only for the easy parts, the scope is in the wrong place.

7. Will This Idea Survive Generative AI?

Generative AI has changed what routine implementation costs. Work that once justified two semesters — building a data-entry application, wiring an API, producing a conventional interface, generating boilerplate and scaffolding — can now be produced quickly with assistance. This does not make final year projects easier. It raises the threshold for what deserves capstone status. An idea whose principal difficulty was the volume of code required is no longer a sufficient idea.

The AI-resilience test is a thought experiment for choosing ideas, not a policy on tools. If a competent final-year student with good tooling could produce a credible working version of the proposed project in a few days, the idea is not yet an FYP. This is not a claim that such a version would be good. It is a signal that the project's difficulty never lay in the implementation. The group should then ask where the remaining difficulty actually is — in the data, the domain, the constraints, the trade-offs, the evaluation or the deployment — and make that the project. If there is no remaining difficulty, the group should choose a different problem.

Ask	What a defensible answer looks like
Could a strong student with AI assistance build a working version of this in a week?	If yes, identify precisely what such a version would get wrong — the data quality, the domain constraints, the edge cases, the failure behavior, the evaluation, the deployment reality — and make that the project. If nothing would go wrong, choose another idea.

Ask	What a defensible answer looks like
What in this project cannot be generated?	Problem formulation, stakeholder study, data acquisition and curation, domain judgement, trade-off decisions, evaluation design, failure analysis, and deployment under real constraints. These are the parts that carry academic weight.
Where does assistance stop being help and start being the project?	If the central capability is supplied by an external model or service, the contribution must lie elsewhere — in the data, the retrieval or pipeline design, the validation, the safeguards, the integration or the evaluation — and the group must be able to state what that is in one sentence.
Can every member explain and modify code they did not write by hand?	Ownership is demonstrated by explanation, modification and defense under questioning, not by authorship of keystrokes. A member who cannot reason about a component does not own it, and should not be credited with it.
Does the evaluation test the system, or only the happy path?	Assistance makes it easy to produce something that demonstrates well and fails quietly. Evaluation should target the cases where the system is most likely to be wrong.
Has the group followed current disclosure policy?	Permitted use of AI tools, generated code, prompts, datasets, models and text must be disclosed and acknowledged exactly as the university requires. Undisclosed use is an integrity matter, not a technical one.

AI raises the bar; it does not lower the standard
The permitted use of AI tools is governed by current university policy and is outside the scope of this guide. What is within scope is the consequence for idea selection: groups should choose problems whose difficulty lies in analysis, data, constraints, trade-offs, evaluation and deployment — the parts of engineering that assistance does not remove. Students remain fully responsible for the reasoning, design, implementation, verification and defense of everything they submit.

8. AI-Assisted Development in an FYP

Industry practice now describes software development along a maturity spectrum. At one end the lifecycle is entirely human-led. In the middle, humans lead both judgment and execution while AI assists with isolated tasks. At the far end — described as AI-driven or agentic development — AI leads execution and humans are repositioned as validators and strategists who approve generated designs, code and tests. The School's position is that a final year project is conducted at the middle level: AI-assisted development, not AI-led development.

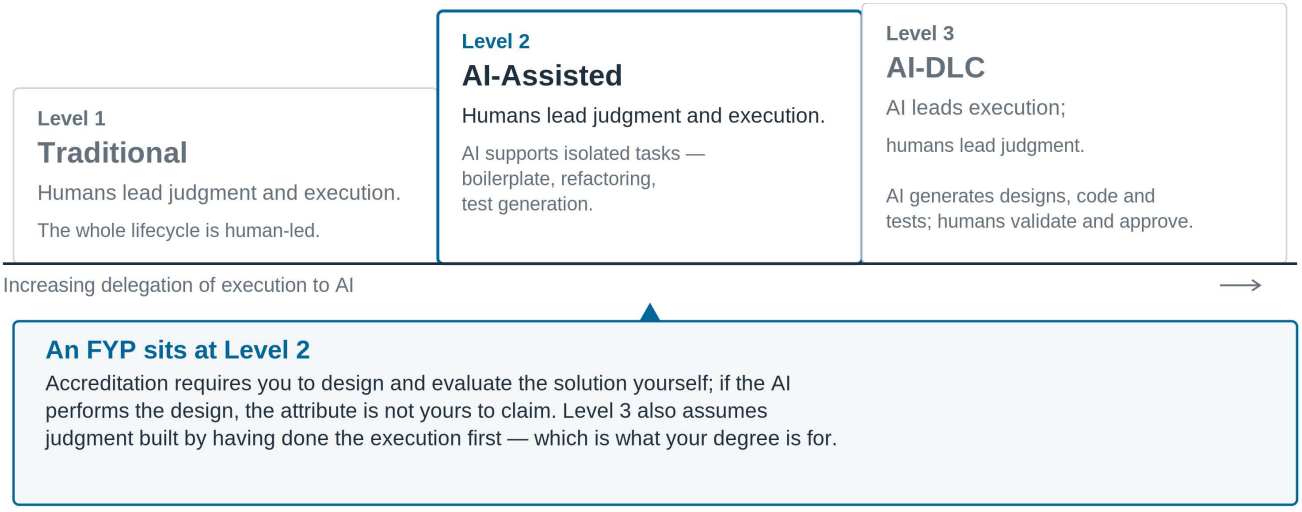


Figure 3. The three levels of development maturity. An FYP is conducted at Level 2.

8.1 Why an FYP is Conducted at the Assisted Level?

Reason	Explanation
Accreditation	Seoul Accord GA-4 requires the graduate to design and evaluate solutions for complex computing problems, and to design and evaluate systems, components or processes. Where an AI performs the design, the attribute has not been demonstrated by the graduate and cannot be reported as attained. GA-2, the abstraction and conceptualization of computing models, fails in the same way.
Judgment follows execution	AI-led development assigns people the validating role — approving designs, reviewing generated code, accepting recommended fixes. Those are senior roles, held by people who can validate because they have previously executed. A student validating work they could not have produced is not exercising judgment. Your degree is where the execution competence that makes judgment possible is built.
The ownership standard	Section 14 requires the member who owns CCP-relevant work to explain it, justify it against an alternative, predict what breaks and modify it under questioning. AI-led development is designed to relocate precisely that capability.
Infrastructure	AI-led development depends on an encoded knowledge base of business rules and architecture, tool integrations across an organization's systems, governance guardrails and production telemetry. None of that exists at project scale, and building it is not the FYP.

Assisted does not mean unassisted

This is not a restriction on using AI tools. Permission is governed by current university policy, use is expected, and permitted use that you disclose is not penalized. It is a statement about where responsibility sits. You may use assistance throughout the project. What you may not do is delegate the judgment your degree exists to develop, or claim ownership of work you cannot account for.

8.2 What You Own and What AI may Assist

The division is not by tool and not by phase. It is by whether the work carries the reasoning a panel will assess. Anything that touches the complex computing problem — how it is formulated, how the solution is designed, which trade-offs were accepted, how it is evaluated and why it fails when it fails — remains yours regardless of what helped you produce it.

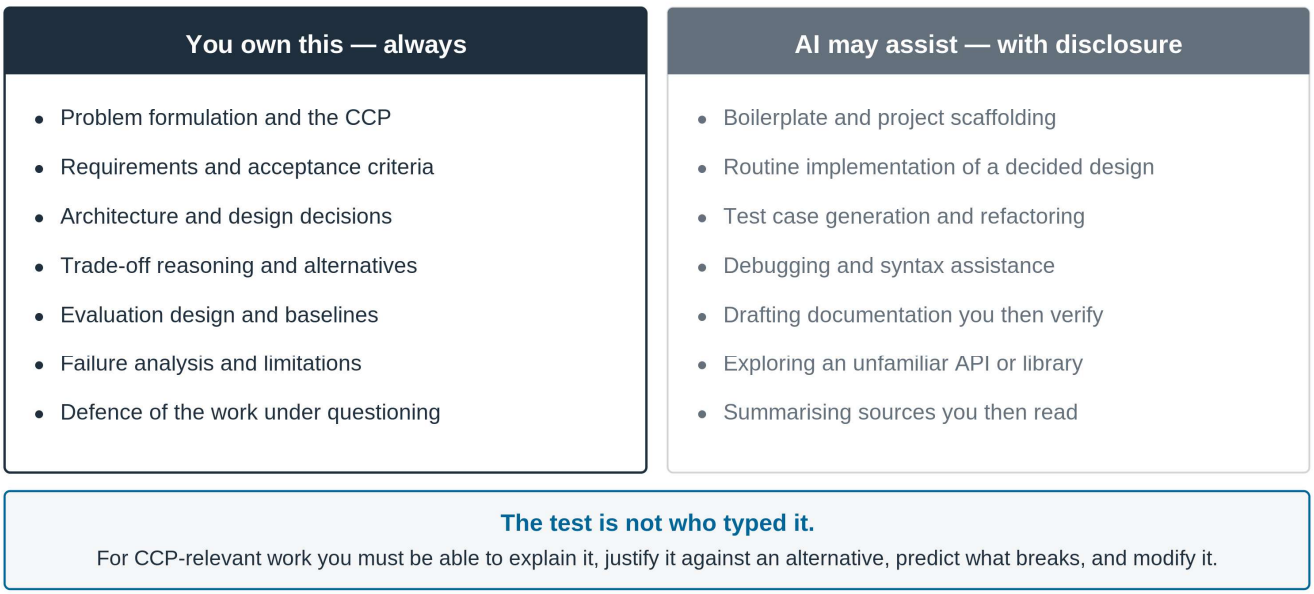


Figure 4. The ownership boundary. Assistance is judged by what you can account for, not by who produced the keystrokes.

Peripheral work is held to a lower standard: you should know what it does and why it is there, but you are not expected to defend every line of routine scaffolding. The distinction matters most at the boundary, so if you are unsure whether a component is CCP-relevant, assume it is and be ready to defend it.

8.3 Specification Before Generation

A consistent finding in AI-native practice is that generated output is only as good as the specification behind it: if the specification is vague, the output will be confidently wrong. A group that can state its requirements, constraints, acceptance criteria and interfaces precisely will get better results from any tool, and will also produce the documentation the project report depends on. A group that cannot will generate plausible code for the wrong problem, and will not notice.

Formulate the problem at requirement level first — what the system must achieve, which constraints cannot be violated, which objectives compete, and what happens on failure — before selecting a method or generating an implementation. Annex C shows what that formulation looks like at three levels of maturity.

8.4 Recording How AI is Used

Disclosure is not a formality and it is not an admission. The School's position is that transparency is more useful than prohibition: a group that records where assistance was used can be assessed on what it understands, while a group that conceals it invites the assumption that it understands nothing. Permitted use that is disclosed is not penalized.

Maintain an AI usage log alongside your dependency disclosure, and update it as you work rather than reconstructing it before a defense. It should record the activity, roughly where in the project it applied, and — for anything touching the complex computing problem — how the output was checked before you relied on it.

Activity	What to record
Ideation and problem exploration	Where assistance shaped the problem framing, the alternatives considered or the literature you went on to read yourself.
Coding assistance and generation	Which components, and whether they are peripheral or CCP-relevant. For CCP-relevant code, state how you verified it behaves as intended.
Debugging and diagnosis	Where assistance located or explained a fault, and whether you confirmed the diagnosis independently.
Test generation	Which tests were generated, and how you checked they test something real rather than restating the implementation.
Documentation and writing	Which artefacts were drafted with assistance and verified by you for technical accuracy before submission.
Anything else	Data cleaning, configuration, refactoring, translation, diagram drafting, or any other use you would not want a panel to discover unrecorded.

The log is evidence for you, not against you
Panels use the log to choose what to ask about, not to deduct marks for using tools. A group that records honest, specific entries and can explain the CCP-relevant ones demonstrates exactly the ownership Section 8.2 requires. An empty log on a project that plainly used assistance, or a log whose entries cannot be defended, is treated as a disclosure failure. The log is submitted with the dependency disclosure sheet and countersigned by your supervisor.

9. A Disciplined Idea-Discovery Process

Do not begin by selecting a technology or asking for a list of trendy applications. Use the following sequence. Several ideas should be explored and rejected before one is taken to proposal defense.

Stream A groups usually enter this sequence at steps 2 and 4, because the literature is where the idea began — but they must still complete steps 1 and 3 by identifying who is affected by the problem and confirming that the gap matters in practice. Stream B groups enter at step 1 and must not skip step 2, because the solution landscape is where most application ideas are discovered to be already solved.

Step	Purpose
1. Explore problem spaces	Identify users, current workflows, failures, consequences and constraints. Speak to relevant stakeholders where feasible.
2. Map the solution landscape	Study commercial products, open-source systems, research prototypes, standards, manual workflows and prior departmental FYPs.
3. Verify the gap	Use evidence to show what remains unsolved or inadequately solved. A missing local version is not automatically a technical gap.
4. Formulate the CCP	Identify which Seoul Accord characteristics genuinely apply, what trade-offs exist, and what knowledge/analysis will be required.
5. Define the contribution	State what the project will add and how it differs structurally from the closest alternatives.
6. Compare technical approaches	Identify alternatives, expected advantages, risks, data/resource needs and the basis for selection.
7. Run a focused POC or technical spike	Test the highest-risk technical assumption before requesting approval.
8. Define evaluation and success criteria	Specify how correctness, usefulness, reliability or improvement will be measured.
9. Shape scope and ownership	Separate core scope from optional work; allocate meaningful primary responsibilities and shared integration duties.
10. Stress-test the idea	Ask what can fail, what may already exist, what depends on unavailable resources, and what evidence would cause the group to revise or reject the idea.
11. Check SDG/thematic alignment	Identify one relevant SDG or thematic area and explain how it connects to the problem and stakeholders without manufacturing importance or complexity.

10. Product, Prior-FYP and Workflow Comparison

Comparison must be structural, not based only on titles or feature lists. Compare the problem, users, inputs, outputs, technical approach, constraints and evaluation. Claims about limitations should be supported by documentation, observed behavior, credible reviews, research evidence or stakeholder input where possible.

Existing solution/ prior FYP	Problem and target users	Core approach/ strong features	Evidence-based limitation	Implication for our project
Solution A				
Solution B				
Closest prior FYP				
Current manual/non-digital workflow				

Important: Changing the platform, language, user interface, model name or market does not by itself make a project different. The group should be able to explain how its problem formulation, constraints, pipeline, contribution or evaluation is materially different.

11. Research-to-Decision Table

Use research to make decisions, not to fill pages. Every important source should have a clear role in the project.

Source/ method/ standard	Main finding or method	Relevance and limitation	Decision influenced	Planned evidence
Source 1				

Source/ method/ standard	Main finding or method	Relevance and limitation	Decision influenced	Planned evidence
Source 2				
Source 3				
Source 4				

12. Proposal-Stage Proof of Concept (POC) or Technical Spike

Every proposal must include at least one focused feasibility proof, and it must be demonstrated at the proposal defense rather than promised for later. The proposal defense is the first formal evaluation stage of the FYP, and the POC is what separates a researched proposal from an attractive intention. This is not a requirement for a polished MVP or a complete product — a small, honest experiment that tests the riskiest assumption is worth considerably more than an elaborate interface. Its purpose is to establish that the greatest technical risk has been examined, and that the group has started investigating the problem rather than describing it.

A group that cannot show a POC at the proposal defense has not yet established feasibility, and its proposal should normally be deferred rather than approved. Where the highest risk is genuinely non-technical — access, permissions, ethical clearance, licensing or data availability — the evidence should address that risk instead, in the same disciplined form: what was assumed, what was tested, what was found, and what changes as a result.

The POC is a condition of approval, not a formality
The panel should ask to see the evidence, not a description of it. A slide stating that a POC was completed is not a POC. The group should be able to show what was run, on what input, with what result, and to say plainly what that result changed in the design, the scope or the plan. A POC that confirmed the assumption is useful; a POC that disproved it and caused the group to revise its approach is usually more valuable, and should be presented as a finding rather than concealed as a setback.

Project risk	Suitable proposal-stage evidence
Data availability or quality	A sample acquisition/cleaning exercise, annotation trial, data profile, privacy check or evidence that the required data can legally and practically be obtained.
Algorithm/model feasibility	A baseline run, small reproduction, sample inference, comparison of candidate methods or analysis of errors on representative cases.
System or integration feasibility	A minimal end-to-end path, protocol/API compatibility test, latency measurement, offline/edge trial or failure/fallback experiment.
Hardware/ IoT feasibility	Sensor reading, communication test, power/range experiment, device integration or small physical prototype.
Security/ privacy feasibility	Threat model, access-control experiment, encryption/performance test, data-flow analysis or proof that sensitive data can be handled safely.
Human/workflow feasibility	Clickable workflow, task walkthrough, early stakeholder feedback, accessibility check or usability risk test.
Deployment feasibility	A test of the intended deployment target: installation on a clean environment, a container build, device provisioning, a platform or store constraint check, or a resource and cost estimate for the environment in which the system is meant to run.

A POC should answer five questions
1. What risky assumption was tested? 2. What input/setup was used? 3. What result was obtained? 4. What was learned or disproved? 5. How will this change the design or plan? A polished interface that does not test a technical or workflow risk is not sufficient by itself.

13. Plan Evaluation Before Approval

Testing, evaluation, and user acceptance are related but different, and a project needs all three. Testing asks whether the implemented system behaves as specified. Evaluation asks whether the proposed solution meaningfully addresses the problem, and how it compares with a baseline, existing approach, benchmark or stakeholder need. Acceptance asks whether the people the system is for will actually use it. This section covers evaluation planning; Section 13.1 sets out the distinction and the acceptance requirement.

Evaluation Question	Examples of Evidence
What is the baseline or comparator?	Existing product/workflow, simple algorithm, previous method, manual process, published result or current practice.
What data, cases or users will be used?	Representative datasets, controlled scenarios, edge cases, user tasks, simulated workloads or field observations.
What metrics or criteria will be measured?	Accuracy, precision/recall, latency, throughput, reliability, security findings, task completion, usability, cost, resource use or acceptance criteria.
What failure cases matter?	Incorrect outputs, unavailable services, noisy inputs, misuse, bias, unsafe recommendations, performance degradation or recovery behavior.
What conclusion can the evidence support?	Define the intended claim narrowly and state limitations. A successful demo alone supports only a limited conclusion.

13.1 Testing, Evaluation, and User Acceptance Are Three Different Questions

Groups routinely conflate these, present one, and believe they have covered all three. They answer different questions and need different evidence.

Question	What it asks	Typical evidence
Testing — does it behave as specified?	Whether the implemented system does what the requirements say, including at the edges and when things fail.	Unit, integration and system tests; edge and failure cases; defect record by severity.
Evaluation — is it any good?	Whether the solution meaningfully addresses the problem, measured against a baseline, comparator, benchmark or stated success criteria.	Baseline comparison, metrics, ablations, failure analysis, stated limits of what the evidence supports.
Acceptance — will the people it is for actually use it?	Whether the intended users or stakeholders, working on realistic tasks, find the system usable, correct and worth adopting.	User acceptance testing with named participants, task outcomes, issues raised and resolved, and a recorded acceptance decision.

User acceptance testing is required for every project and is evidenced at FYP-2 Final; the plan for it must exist by FYP-2 Mid so that it is not attempted in the final week. Plan it at proposal stage: knowing who will accept the system usually changes what you build.

What your acceptance testing must record	Notes
Method	How testing was run — supervised task sessions, field trial, pilot deployment, structured walkthrough or expert review — and why that method suits the system.
Participants	How many, and who they are in relation to the intended users: real end users, operators, domain experts or proxies. State plainly which. Numbers are usually small; that is acceptable if the participants are the right ones and you do not over-claim from them.

What your acceptance testing must record	Notes
Tasks and results	The tasks participants attempted, what succeeded, what failed, and any measures taken — completion, time, errors, assistance needed, satisfaction.
Issues found and what you did	Problems raised, which were fixed before final submission, which were deferred, and why. Issues found and fixed are evidence of a working process, not of failure.
Acceptance outcome	Whether the stakeholder accepts the system for its intended use, with any conditions or reservations recorded in their words rather than yours.
Recording	A short video of a representative session, with participant consent. It is evidence that the testing happened as described and lets examiners verify outside panel time.

Where real users cannot be reached
Some systems cannot be tested with their intended users inside a two-semester project — clinical tools, safety-critical systems, security tooling, or research artefacts with no direct operator. The requirement does not disappear; the participant definition changes. Use domain experts, operators, or the closest defensible proxy, declare the substitution at proposal stage rather than at the final defense, and state clearly what an acceptance result from proxies can and cannot tell you. Consent, privacy and any institutional approval needed to involve participants are the group's responsibility and should be resolved early.

14. External Components and Student Ownership

Modern systems normally use frameworks, libraries, APIs, pretrained components, templates and other development resources. Their presence is neither a strength nor a weakness in itself. What matters is what a component supplies relative to the contribution the group is claiming.

Every important external component falls into one of three categories. The same three categories appear on the External Assistance and Dependency Disclosure Sheet in the evaluation forms, so a group that classifies its dependencies at idea stage will already have done most of the disclosure work it must submit later.

Category	What it means	What is required
Support	Tooling, infrastructure, scaffolding, boilerplate, standard libraries, UI kits, build and deployment tooling, and anything else that does not touch the claimed contribution.	Disclosure and licence compliance. No further justification is expected, and panels will not spend time here.
Core-assist	A component that contributes to a part of the system relevant to the complex computing problem — a hosted model, a service API, a published implementation, or a template that shapes a central workflow.	Permitted. The group must justify the choice, state the component's limitations, validate its outputs, plan a fallback, and be able to explain and modify the surrounding design. The contribution must remain identifiable independently of the component.
Core-replacement	A component that supplies the contribution itself: the complex computing problem is solved by the component rather than by the group.	Not acceptable. Either the claimed contribution is redefined around what the group actually does, or the component is replaced by the group's own work. This is a scoping decision and should be taken before approval, not discovered at defense.

The boundary is the contribution, not the technology
A group may use a hosted model, a published implementation or a commercial API and still produce an excellent FYP — provided it can say in one sentence what remains its own. Apply the test in Section 4 directly: if the external component were replaced by a comparable alternative, would the claimed contribution survive? If the answer is no, the component is the contribution, and the project needs rescoping rather than defending.

14.1 The Standard of Understanding for CCP-Relevant Work

Panels do not ask who typed a line of code. They ask whether the member who claims a component can account for it. For any part of the system that touches the complex computing problem, the responsible member should be able to do four things.

The responsible member should be able to	What a panel will ask
Explain what it does and why it is there	Walk through the component's role in the system and the sub-problem it addresses, without reading from the code.
Justify why it is built or configured this way	Name at least one alternative that was considered, and say why it was rejected.
Predict what breaks	State what fails if an input, assumption, dependency, requirement or scale changes — and what the system does when it fails.
Modify it under questioning	Describe, or make, a specific change and identify what else must change with it.

A member who cannot do these four things does not own that component, whatever the repository history shows. The standard applies uniformly: it does not matter whether the code was written from scratch, adapted from a tutorial, taken from a published implementation, or produced with assistance. Peripheral code is held to a lower standard — the group should know what it does and why it is there, but is not expected to defend every line of routine scaffolding.

Two further points. Ownership is individual. The obligation attaches to the member who claims the responsibility, and the team as a whole should understand the interfaces between components even where only one member understands the internals.

Classification can change. If the group's complex computing problem claim shifts during the project — which Section 5.3 permits when the change is recorded — a component previously classified as support may become core-assist. The disclosure sheet should be updated when that happens, not reconstructed at the end.

Questions to ask about every important dependency:

- What exactly does the external component provide, and which part remains the students' technical responsibility?
- Would the proposed FYP still contain a meaningful computing contribution if that component were replaced?
- How will outputs, failures, limitations, cost, security and fallback behavior be tested?
- Can the group explain and modify the surrounding design and integration?
- Has the group correctly cited or attributed external code, data, models, prompts, text and documentation where required?

15. Common Weak Idea Patterns

Weak pattern	Why it is weak	What would make it defensible
Routine portal, CRUD system or feature-rich application	Size and screen count do not establish a CCP or contribution.	Identify a genuine technical/workflow problem, design trade-offs and evaluation beyond implementation.
Generic chatbot or retrieval wrapper	The central capability may be provided by an external model/service.	Define a domain problem, retrieval/data design, validation, safeguards, failure analysis and contribution beyond the service.
Model training with no problem or baseline	A model and an accuracy value do not show useful problem solving.	Define the use case, dataset quality, baseline, metrics, practical constraints, error analysis and system integration.
Local copy of an existing product	Localisation alone is usually not a computing contribution.	Show a local constraint that changes requirements, architecture, data, method or evaluation.

Weak pattern	Why it is weak	What would make it defensible
Implementation of a paper without understanding	Reproduction can be valuable, but copying steps is not enough.	Explain the method, reproduce transparently, compare results, analyse differences and adapt or integrate it meaningfully.
Independent member modules with weak integration	The project lacks coherence and shared system responsibility.	Use one central problem, defined interfaces, integrated evaluation and shared architecture/quality responsibilities.
Industry feature delivery only	Client satisfaction does not by itself show academic learning.	Map the work to problem analysis, design decisions, testing, evaluation and individual technical ownership.
SDG/ thematic label without problem connection	A broad theme or SDG label does not prove technical depth, stakeholder need or contribution.	Explain the real problem and stakeholders first, then show a natural and limited SDG/thematic connection.
Project whose only difficulty is the volume of code	Scale of implementation no longer establishes capstone-level difficulty, because routine code is now cheap to produce.	Relocate the difficulty into problem formulation, data, constraints, trade-offs, evaluation or deployment, and state where it now sits.
Paper reproduced with no system (Stream A)	A result in a notebook is not an FYP artefact; the standard requires a system that can be deployed and operated.	Engineer the method into an operable system with an identified user, a stated deployment target and an evaluation on realistic input.
Application built with no research basis (Stream B)	A working product does not by itself demonstrate research-informed problem solving, a CCP or a defensible contribution.	Establish the evidence base, identify the complex computing problem in the constraints, and define the evaluation before adding features.
System that runs only on the developer's machine	Deployability is part of the core standard; a system that cannot be installed and operated by anyone else is incomplete.	Define the deployment target early and test installation, configuration and operation by someone outside the team.

16. Proposal Defense Package

The proposal presentation should be concise and should present evidence before the panel begins questioning. The slide order may be adjusted, but all of the following information should be visible.

Suggested slide	Required content
1. Title and one-line problem	Project title, members, supervisor, declared stream (research-derived or application-derived) and a one-sentence problem statement.
2. Stakeholders and current workflow	Who faces the problem, how it is currently handled, why the failure matters, and the one relevant SDG or thematic area that gives broader context.
3. Existing solutions and prior FYPs	Structural comparison and evidence-based gap.
4. Research and technical basis	Relevant current/authoritative work, alternatives studied, and decisions influenced.
5. CCP/ complex activity justification	Applicable Seoul Accord characteristics, central trade-offs, uncertainty and why routine development is insufficient.
6. Proposed contribution and solution	Contribution statement, high-level architecture, major components, boundaries and the intended deployment target.
7. POC/ technical spike (mandatory)	The risk that was tested, the evidence obtained — shown, not described — what was learned or disproved, and the resulting design or plan decision. A proposal that cannot demonstrate feasibility evidence should not normally be approved.
8. Evaluation plan	Baseline, data or scenarios, metrics, success criteria, at least one measurable non-functional criterion, and the failure cases that matter most.

Suggested slide	Required content
9. Scope, risks and dependencies	Committed scope, optional extensions, resources, data, modern tools and external dependencies, professional and ethical compliance, deployment and integration risk, and the fallback plan.
10. Team responsibilities and iterations	Primary technical ownership, shared integration/quality work and FYP-1/FYP-2 milestones.

17. Final Idea-Readiness Checklist

[ ] We can state the problem, users, current workflow, gap and contribution in one minute.

[ ] We have identified one relevant SDG or thematic area and can explain its connection without using it as artificial complexity.

[ ] Our gap is supported by comparison with products, open-source systems, prior FYPs, research or current practice.

[ ] Our research/technical sources influence specific design or evaluation decisions.

[ ] We have identified the central CCP and can justify the applicable Seoul Accord characteristics.

[ ] The complexity arises from the real problem, not from adding technologies or features.

[ ] Our contribution is meaningful even if it is reproduction, adaptation, evaluation or constrained system design rather than new algorithmic novelty.

[ ] We have a focused proposal-stage POC or technical spike that tests the assumption carrying the greatest risk, and we can demonstrate the actual evidence at the defense rather than describe it.

[ ] We can state what our POC confirmed or disproved, and exactly what it changed in our design, scope or plan.

[ ] If we are implementing or improving a state-of-the-art method, we can explain its mechanism, assumptions and failure modes without the code in front of us, and we can state our baseline and our delta.

[ ] We have defined a baseline, success criteria, evaluation evidence and important failure cases.

[ ] Every important external component is classified as support, core-assist or core-replacement, and nothing we claim as our contribution sits in the core-replacement category.

[ ] For every part of the system that touches our CCP, the responsible member can explain it, justify it against an alternative, predict what breaks if it changes, and modify it under questioning.

[ ] We have disclosed and acknowledged permitted external code, AI tools, licensed data, copyrighted material, datasets, models, APIs and templates according to current policy.

[ ] The project is one coherent system with meaningful primary technical ownership for all members and shared integration responsibility.

[ ] We have separated committed scope from optional extensions and can explain what should be complete in FYP-1 and FYP-2.

[ ] We know who will accept this system, how acceptance will be tested, and whether they are real users or declared proxies.

[ ] We can state where the system will be deployed for real, and what would prevent that deployment if anything would.

[ ] We are keeping an AI usage log as we work, and can defend every CCP-relevant entry in it.

[ ] Every member can explain the complete idea and defend at least one important technical decision.

[ ] We understand data, legal, ethical, security, privacy, cost, deployment and dependency risks relevant to the project.

[ ] We have checked structural overlap with previous departmental FYPs, not only title similarity.

[ ] We can answer: what will still require serious analysis, engineering and evaluation after routine implementation work is complete?

[ ] We have declared our stream and can explain whether the idea entered through published research or through an observed real-world problem.

[ ] We can state where the system will run when deployed, and who besides us could install and operate it from our documentation.

[ ] We can name concrete functional requirements, at least one measurable non-functional requirement, and an acceptance criterion for our hardest requirement.

[ ] We have asked whether a competent student with AI assistance could produce a credible version of this quickly, and we can say where the remaining difficulty lies.

Final decision rule
Do not select an idea because it is fashionable, easy to demonstrate or rich in features. Select it when the group can defend the problem, research basis, complex computing challenge, contribution, feasibility, evaluation and ownership as one coherent story.

Annex A — Stream A: Starting from Published Research

Stream A projects begin from published work: a method, a benchmark, a dataset or an open problem stated in the literature. The strength of this entry point is that the technical foundation is already articulated and the evaluation vocabulary already exists. Its characteristic weakness is that a group can spend two semesters re-running someone else's code and arrive with neither a system nor an understanding. This annex addresses that risk. It supplements the common requirements in Sections 1 to 17 and does not replace any of them.

A.1 Choosing the Research Paper

The choice of paper determines most of the project's risk. Make it deliberately, and test the riskiest assumption before requesting approval — not in the fifth month.

Criterion	What to check before committing
Standing and recency	Prefer current peer-reviewed work from credible venues, supplemented by seminal papers and official standards where the foundations matter. A single unreviewed preprint with no independent validation is a weak basis for two semesters of work.
Reproducibility	Are the method, parameters, data and evaluation protocol described well enough to reproduce? Is code or data released? If not, the reproduction itself becomes the principal risk and must be tested at proposal stage.
Resource feasibility	Can the method be trained, fine-tuned or run within the compute, storage, license, budget and time actually available to the group? Verify this in the proposal-stage spike, and identify a fallback method before approval.
Data availability	Is the dataset obtainable, legally usable and representative of the intended deployment context? Data that the group must collect or annotate is a project in itself and must be scoped, resourced and ethically cleared as such.
Room for contribution	Does the paper leave a stated limitation, an untested setting, a missing comparison, an unexamined assumption or an unaddressed constraint? A paper with no visible opening leaves the group with nothing to add.
Path to a system	Can the method plausibly become something a user or operator can run? If the answer is no, the idea fails the deployability requirement in Section 1.2 regardless of its research merit.

A.2 Establishing Your Delta

The delta is what the project adds beyond the paper. It need not be novelty in the sense of invention. A rigorous reproduction, a well-argued adaptation, a fair comparison or serious engineering under constraints are all legitimate — provided the group states clearly what it is doing and evaluates it honestly.

Type of delta	What must be shown
Faithful reproduction with transparent reporting	Where published results are difficult to reproduce, careful reproduction is a legitimate contribution — provided the group reports the protocol, every difference from the original, the results obtained, and a reasoned analysis of any divergence.
Adaptation to a new context	A different domain, language, population, sensor, data regime or regulatory setting — with evidence that the change is non-trivial and an analysis of what it does to performance and to the method's assumptions.
Constraint-driven engineering	Running the method under a constraint the paper did not consider: latency, memory, offline operation, edge hardware, cost, energy, privacy or continuous operation. State the constraint as a number, not as an aspiration.
Comparison and benchmarking	A fair, documented comparison of candidate methods on a problem or dataset where no such comparison exists, with an honest account of what the comparison can and cannot support.
Failure and limitation analysis	Systematic characterization of where the method breaks, on which inputs and why — particularly valuable where the original evaluation was narrow or the reported setting was favorable.
Integration into an operable system	Engineering the method into a deployable system with real inputs, real users, real latency and real failure handling. In most Stream A projects this is the largest and most defensible contribution.

State the delta in one sentence
"Paper X reports result A on dataset B under assumption C. We will reproduce, adapt or extend it to D, where assumption C does not hold, and evaluate using E." If the group cannot complete this sentence, the paper has not yet been understood well enough to build on. If the sentence is true but trivial, the delta is too small for two semesters.

A.3 From a Result to a System

A reproduction that lives in a notebook does not satisfy the School's standard. Stream A groups must plan from the beginning how the method becomes a system: what the inputs are in real use, who supplies them, what happens when the model is wrong, slow or unavailable, how the system is deployed and monitored, and what an operator actually sees. This engineering work is not a lesser activity appended to the research. In most Stream A projects it is precisely where the complex computing problem lives — in the trade-offs between accuracy and latency, capability and cost, automation and human oversight, and in the gap between a curated benchmark and messy real input.

A.4 Common Stream A Hazards

Hazard	How to avoid it
Running released code without understanding it	Require each member to explain the method's core mechanism, its assumptions and its failure modes without reference to the code. If they cannot, the group has not yet started the project.
Choosing a paper that cannot be run on available resources	Test the heaviest resource assumption in the proposal-stage spike, and have a documented fallback method identified before approval is requested.
Selecting a method the group cannot explain	Working with recent, high-performing methods is encouraged, and implementing or improving one is a strong basis for an FYP. Selecting one the group cannot explain is not. A well-understood recent method is the strongest position; a well-understood older method is a better project than a poorly understood recent one.
Treating the metric as the goal	An improvement in a benchmark figure is not automatically an improvement for a user. Connect every metric to the stakeholder consequence it is standing in for.

Hazard	How to avoid it
Benchmark data that does not resemble real input	Test the method early on data that looks like what the deployed system will actually receive. Degradation on realistic input is a finding worth reporting, not a failure to hide.
Leaving the system to the last month	Plan the end-to-end path early, even crudely, so that integration and deployment risk surfaces in FYP-1 rather than in the closing weeks of FYP-2.

Annex B — Stream B: Starting from a Real-World Problem

Stream B projects begin from something observed: a workflow that fails, a task done manually at unnecessary cost, a need that no available product serves well. The strength of this entry point is that the problem is real and the users exist. Its characteristic weakness is that a group builds a competent application around a problem that never required computing depth, and then finds at defense that it cannot identify a complex computing problem. This annex addresses that risk. It supplements the common requirements in Sections 1 to 17 and does not replace any of them.

B.1 Validating that the Problem is Real

A problem is not established by assertion. Before an idea is defended, the group should have evidence that the problem exists, that it matters, and that it is not already adequately solved.

Evidence	What counts
Stakeholder contact	Conversations, observation, interviews or a small structured survey with people who actually face the problem. Assumed users are not stakeholders. Record who was consulted, when, and what was learned.
The current workflow	A concrete description of how the task is done today, including its cost in time, money, error or risk. If the current workflow cannot be described, any improvement over it cannot be measured.
Consequence of failure	What goes wrong when the current approach fails, and to whom. This determines how seriously the proposed system must be tested, safeguarded and evaluated.
Why existing products do not serve it	A structural comparison against real products, not a dismissal by feature list. “It is not available here” and “it is expensive” are market observations rather than technical gaps — though either may create a genuine constraint worth engineering around.
Constraints that are not negotiable	Connectivity, cost, device capability, literacy, language, regulation, privacy, or existing systems that must be integrated with and cannot be changed. These constraints are usually where the computing complexity comes from.

B.2 Finding the Complex Computing Problem

The most common Stream B failure is a good problem with a shallow solution. The remedy is not to add technology; adding technology produces artificial complexity, which Section 3 explicitly warns against. The remedy is to look harder at the constraints. Complexity in application-derived projects rarely comes from the features. It comes from what the real environment refuses to allow.

Source of genuine complexity in application projects	What it looks like
Conflicting stakeholder requirements	The regulator, the operator and the end user want incompatible things, and the design must resolve the conflict explicitly rather than quietly favor one party.
Hard environmental constraints	Intermittent connectivity, low-end devices, limited power, high-latency links, offline-first operation, or integration with a system whose behavior cannot be changed.

Source of genuine complexity in application projects	What it looks like
Data that is messy, scarce, sensitive or contested	Incomplete records, inconsistent formats, personal or regulated data, multiple sources that disagree, or data that must be acquired and curated before anything can be built.
Correctness with real consequences	Decisions affecting health, safety, money, legal standing or access to a service — requiring validation, auditability, human oversight and defensible failure behavior.
Scale or timing behavior	Concurrency, real-time deadlines, large volumes, or workloads that make the straightforward implementation unusable in practice.
Interoperability	Systems, protocols, standards or organisations that must be made to work together where no single party controls the whole.
Trust, security and privacy	Adversarial users, sensitive data, access control across roles or organizations, or a requirement to prove afterwards what the system did and why.

If the constraints disappeared, would the problem disappear?
Ask: if connectivity were perfect, the data were clean, every user were an expert and failure had no consequence — would this still be a hard computing problem? If not, then the constraints are the project, and they must be stated, designed for and evaluated explicitly rather than mentioned in passing. If nothing hard remains under any conditions, the idea is not yet an FYP.

B.3 What Stream B Groups Most Often Need to Add

Requirement	What is usually missing
Research basis	A study of comparable products, research literature, standards and prior FYPs that visibly changes the design — not a literature chapter written after the system was built.
A measured baseline	The current manual or existing digital workflow, measured. Without it there is nothing to compare against and no way to demonstrate improvement.
Evaluation beyond demonstration	Task-completion studies, measured performance against the baseline, error and failure analysis, or stakeholder evaluation on real tasks. A working demonstration supports only a very limited conclusion.
Explicit non-functional targets	Latency, availability, security posture, accessibility standard, offline behavior or data protection, stated as measurable criteria and tested.
Technical depth in at least one subsystem	At least one component whose design required genuine analysis and trade-off, which the group can defend in detail under questioning.

B.4 Common Stream B Hazards

Hazard	How to avoid it
Feature accumulation	Adding modules, roles and dashboards to create the appearance of scope. Commit to a narrow core and deepen it; move everything else to clearly separated optional extensions.
Assumed users	Building for an imagined stakeholder. Speak to real ones early, and record what changed in the requirements as a result of doing so.
The “local version” trap	Rebuilding an existing product for a local market. This is defensible only when a local constraint genuinely changes the requirements, architecture, data or evaluation — and the group must say which.
Interface treated as contribution	A usable interface is expected, not claimed as a contribution. The contribution must lie in the problem, the design, the data or the evaluation.

Hazard	How to avoid it
Ethics and data access discovered late	Consent, permissions, personal data handling and institutional approval should be resolved at proposal stage. Projects that depend on access nobody has agreed to are high-risk by construction.
Late discovery that there is no complexity	Identify and state the complex computing problem in the proposal, not after the application has been built. If it cannot be found at proposal stage, it will usually not appear later.

Annex C — Three-Level CCP Formulation Examples

The hardest part of idea selection is not understanding the definition of a complex computing problem; it is recognising the difference between a formulation that sounds ambitious and one that actually is. This annex shows twelve domains, each formulated at three levels of maturity. Reading the contrast is usually more useful than re-reading the characteristic table in Section 3.

Examples C.1 to C.6 are application-derived (Stream B). Examples C.7 to C.12 are research-derived (Stream A). The domains are illustrative only: they are not a menu of approved projects, and a group that adopts one of these problems still has to establish its own evidence, stakeholders, gap and contribution.

The three levels are not arbitrary — they follow the Seoul Accord
Seoul Accord Section D.4.1 defines three tiers of computing problem. A Well-defined problem "can be solved in standardized ways" and is "encompassed by standards and/or documented procedures of practice". A Broadly-defined problem "can be solved by application of well-proven analysis techniques" and "belongs to families of familiar problems". A Complex problem "has no obvious solution, and requires conceptual thinking and innovative analysis", and "is outside problems encompassed by standards and standard practice". The Weak, Developing and Strong levels below correspond to those three tiers. This matters: The Accord scopes Well-defined problems to the computing technician and Broadly-defined problems to the computing technologist, and reserves Complex problems for the computing professional — the graduate an accredited BS program produces. A Developing formulation is therefore not simply a less ambitious FYP; it sits below the level the accreditation expects of your degree.

The Strong formulations share a structure worth noticing. Each states the required outcome, the constraints that cannot be violated, the objectives that compete, the uncertainty the system must survive and the consequence of getting it wrong — and each does so before naming any method, model, framework or architecture. Formulating the problem at requirement level first is what makes the difficulty visible. A group that begins by choosing a technology has usually already decided the answer and will struggle to explain what made the question hard.

Application-derived examples (Stream B)

C.1 University Timetabling Product

Level	CCP/ problem statement
Weak	Develop an automated university timetable system that allocates courses, faculty members and rooms while avoiding clashes.
Developing	Develop a timetable-generation system that considers faculty availability, room capacities, laboratory requirements, student course clashes, instructor preferences and preferred teaching hours.
Strong	Develop a timetable-generation and repair system that must satisfy zero faculty, room and student-batch clashes; laboratory, equipment, accessibility and restricted-availability constraints; and must balance compact student schedules, fair distribution of undesirable hours, instructor preferences, room utilisation and minimum disruption when late changes make all preferences impossible to satisfy simultaneously.

C.2 Institutional Information Assistant

Level	CCP/ problem statement
Weak	Develop a RAG-based university chatbot that answers questions from university documents.

Level	CCP/ problem statement
Developing	Develop a university assistant that processes policies and handbooks, provides citations, supports multiple user roles and updates its knowledge base when documents change.
Strong	Develop a university information assistant that must answer from documents differing in authority, effective date and access level; distinguish current from superseded rules; prevent restricted-information disclosure; identify conflicting or insufficient evidence; and balance answer completeness, response time and the risk of unsupported responses.

C.3 Fraud Investigation Product

Level	CCP/ problem statement
Weak	Develop an AI-based system that detects fraudulent transactions and displays alerts on a dashboard.
Developing	Develop a fraud-monitoring product that combines transaction, account, device and channel history to calculate risk scores and prioritise alerts.
Strong	Develop a fraud-investigation product that must identify coordinated and evolving behavior across accounts, devices, channels and time while controlling false positives, preserving privacy, producing traceable explanations, prioritizing limited investigator capacity and meeting near-real-time response constraints.

C.4 Coordinated Traffic-Control Product

Level	CCP/ problem statement
Weak	Develop a smart traffic system that detects vehicles and changes signal timings.
Developing	Develop a traffic-management system for multiple intersections that considers vehicle volume, pedestrians and emergency vehicles.
Strong	Develop a coordinated traffic-control product that must preserve pedestrian and intersection safety, prioritize emergency vehicles, balance waiting time across competing traffic streams, prevent local optimization from worsening network congestion and continue operating under uncertain flows, sensor errors and communication delays.

C.5 Recruitment Decision-Support Product

Level	CCP/ problem statement
Weak	Develop an AI-based recruitment platform that ranks CVs against job descriptions.
Developing	Develop a recruitment product that evaluates education, experience and skills, ranks applicants and provides explanations to recruiters.
Strong	Develop a recruitment decision-support product that must reason from incomplete and inconsistently presented evidence, balance conflicting job criteria, control unfair exclusion and proxy bias, preserve auditability, support justified human overrides and communicate uncertainty where candidate evidence is insufficient.

C.6 Environmental-Risk Monitoring Product

Level	CCP/ problem statement
Weak	Develop an IoT system that displays sensor readings and sends alerts when values cross thresholds.
Developing	Develop an environmental monitoring product that combines readings from multiple sensors, reports trends and detects abnormal conditions.

Level	CCP/ problem statement
Strong	Develop an environmental-risk monitoring product that must distinguish hazardous events from sensor drift, failure and communication loss; combine heterogeneous observations of different reliability; preserve timely alerts under power and bandwidth limits; and balance missed-event risk against false alarms that cause unnecessary intervention.

Research-Derived Examples (Stream A)

In a research-derived project the published work normally provides the starting point. The Strong formulation must still identify the substantial problem that remains the group's own, and the operating requirements of the product the contribution will sit inside.

C.7 Diagnostic-Support Product (Medical Imaging)

Level	CCP/ problem statement
Weak	Reproduce a published deep-learning model for medical-image classification.
Developing	Apply and compare published models on a local dataset and improve classification accuracy.
Strong	Develop a diagnostic-support product that must operate on heterogeneous, low-quality and partially missing field images; recognize distribution shift and uncertain cases; control clinically dangerous false negatives; provide traceable evidence; and operate within the compute and connectivity limits of the intended setting.

C.8 Urdu Opinion-Analysis Product

Level	CCP/ problem statement
Weak	Fine-tune a published language model for Urdu sentiment analysis.
Developing	Develop an Urdu sentiment model that handles Roman Urdu, code-switching and spelling variation.
Strong	Develop an Urdu opinion-analysis product that must handle code-switching, Roman Urdu, sarcasm, evolving vocabulary, scarce and inconsistent labels, domain shift and unequal performance across user groups while providing interpretable evidence for high-impact moderation or decision-support cases.

C.9 Federated Learning Product

Level	CCP/ problem statement
Weak	Implement federated learning using an existing framework and default aggregation.
Developing	Train a shared model across multiple organizations without exchanging raw data and compare it with centralized training.
Strong	Develop a federated learning product that must maintain useful and fair performance across highly non-identical institutional data, intermittent participation, unequal client resources, communication limits, privacy threats and unreliable client updates without exposing local data.

C.10 Misleading-Chart Review Product

Level	CCP / problem statement
Weak	Implement a published model for detecting misleading charts.
Developing	Detect multiple chart types and classify common misleading visual patterns.

Level	CCP / problem statement
Strong	Develop a chart-review product that must analyse charts extracted from heterogeneous images, PDFs and dashboards; reason with incomplete metadata and surrounding context; distinguish design error from intentional distortion; control costly false accusations; explain supporting evidence; and route uncertain cases for human review.

C.11 Legal Information Product

Level	CCP/ problem statement
Weak	Fine-tune a pre-trained language model for legal question answering.
Developing	Develop a legal question-answering system that retrieves jurisdiction-specific documents and provides citations.
Strong	Develop a legal information product that must distinguish binding, advisory, superseded and jurisdiction-specific sources; handle conflicting authorities and changing law; prevent restricted-document disclosure; identify insufficient evidence; and balance answer usefulness with the consequences of unsupported guidance.

C.12 Operational Intrusion-Analysis Product

Level	CCP/ problem statement
Weak	Reproduce a published intrusion-detection model on a public benchmark dataset.
Developing	Compare several intrusion-detection models and improve detection accuracy on local network data.
Strong	Develop an operational intrusion-analysis product that must detect evolving and low-frequency attacks from imbalanced, partially labelled and changing traffic; control alert overload; preserve response time; remain robust to adversarial manipulation; provide evidence suitable for investigation; and adapt without forgetting previously detected behaviors.

C.13 Which Characteristics each Example Exhibits?

The numbers below refer to the nine Seoul Accord characteristics listed in Section 3. Every Strong formulation exhibits at least six of the nine, and all twelve exhibit characteristics 1, 2, 3 and 5 — including the two that most clearly separate a complex problem from a routine one: no obvious solution, and beyond standard practice. None exhibits all nine by default, which is the point: the Accord requires “some or all”, not a full set.

Example (Strong formulation)	Characteristics clearly exhibited	Present only weakly
C.1 University timetabling	1, 2, 3, 5, 6, 8	4, 7, 9
C.2 Institutional information assistant	1, 2, 3, 4, 5, 6, 8, 9	7
C.3 Fraud investigation	1, 2, 3, 4, 5, 6, 7, 8, 9	—
C.4 Coordinated traffic control	1, 2, 3, 4, 5, 6, 7, 8	9
C.5 Recruitment decision support	1, 2, 3, 4, 5, 6, 7, 8, 9	—
C.6 Environmental-risk monitoring	1, 2, 3, 4, 5, 7, 8, 9	6
C.7 Diagnostic support (imaging)	1, 2, 3, 4, 5, 7, 8, 9	6

Example (Strong formulation)	Characteristics clearly exhibited	Present only weakly
C.8 Urdu opinion analysis	1, 2, 3, 4, 5, 7, 9	6, 8
C.9 Federated learning	1, 2, 3, 4, 5, 6, 8	7, 9
C.10 Misleading-chart review	1, 2, 3, 4, 5, 7, 9	6, 8
C.11 Legal information	1, 2, 3, 4, 5, 6, 7, 8, 9	—
C.12 Operational intrusion analysis	1, 2, 3, 4, 5, 7, 8, 9	6

Read the third column as carefully as the second
Characteristic 6, diverse stakeholders, is clearly present in only seven of the twelve examples — the most commonly missing characteristic in the set. Technically strong formulations often under-state who else has a claim on the system's behavior, and panels should probe it. A group whose problem genuinely has one stakeholder group should say so plainly rather than invent others; a group that has simply not looked should look again before defending.

C.14 How to Use These Examples

DOs	DON'Ts
Use the contrast diagnostically: write your own formulation, then decide honestly which of the three columns it most resembles.	Do not adopt a domain from this annex because it appears here. These are illustrations of formulation quality, not pre-approved project titles.
Notice that every Strong statement names competing objectives and a failure consequence. If yours names neither, it is probably a Developing formulation.	Do not manufacture constraints to reach the Strong column. An invented constraint that the evaluation never tests is artificial complexity, which Section 3 warns against.
Formulate at requirement level first, and let the method, architecture and technology follow from the constraints you identified.	Do not treat the Strong statement as a scope commitment. It states the problem; Section 5 governs how much of it a group commits to across two semesters.
Expect the panel to ask which characteristics apply to your project and what project-specific evidence supports each claim.	Do not claim characteristics you cannot evidence. A short, honest list of four well-supported characteristics defends better than nine asserted ones.

References

- Seoul Accord, Section D: Graduate Attributes, especially D.4 (range of problem solving and computing activities) and D.5 (problem analysis, design/development, modern tool usage, teamwork and communication).
- National Computing Education Accreditation Council (NCEAC), Accreditation Manual v2.1, especially Sections 3.2.2 and 3.2.3.5. The manual describes FYDP as a two-semester capstone mainly involving literature search, individual analysis, formulation of a complex problem and its solution through complex computing activity.
- Higher Education Commission (HEC), Revised Curriculum for BS Computer Science (2025), especially Annex-B: Final Year Project Guidelines for Computing Programs. The guideline requires alignment with relevant SDGs or thematic areas, clearly defined student roles, stage-wise evaluation through proposal, progress reviews, final demonstration and report submission, appropriate final deliverables, and compliance with ethical, plagiarism, licensing, copyright and AI-tool acknowledgement requirements.
- FAST School of Computing's existing FYP process, proposal defense SOP, evaluation forms and practices, together with feedback from faculty supervisors, evaluators, the FYP committee and academic leadership during the review of the FYP process.
- The School of Computing's project documentation standard — scope of work, proposal and initiation plan, timeline of milestones and deliverables, software requirements specification, software design specification, requirements traceability and acceptance matrix, and test, deployment and handover plan — together with its five approval gates, which define the written deliverables that follow idea approval and together form the project report.

- Industry practice on AI-assisted and AI-driven development lifecycles, including the AI-Driven Development Life Cycle (AI-DLC) described by Amazon Web Services and related commentary on development-maturity levels. These sources inform the maturity spectrum in Section 8. They describe commercial practice rather than accreditation requirements, and where they conflict with Seoul Accord graduate-attribute obligations, the latter govern.